

Learning-Augmented Disk Scheduling: A Lightweight Scheduler Selector under Workload Shift

Lt MD Rubiat Hasan Raiyan
202314201
CSE-307 Operating Systems, Section B
October 2026

1 Introduction and Problem Framing

One of the key jobs of an operating system is disk scheduling — deciding the order in which I/O requests get served. The way the disk head moves can make a huge difference in overall performance. Even though most modern storage is SSD-based, the classic disk scheduling problem remains a great way to understand trade-offs between simplicity, seek time, and fairness.

In this project, I implemented four well-known disk scheduling algorithms: First-Come-First-Served (FCFS), SCAN, C-SCAN, and Shortest Seek Time First (SSTF). Each algorithm makes different assumptions about the workload. FCFS is straightforward but ignores locality. SSTF aggressively goes after the closest request, which can lead to starvation in some cases. SCAN and C-SCAN try to reduce unnecessary head movement by sweeping across the disk in an organized way.

The main question I wanted to explore was whether a lightweight learned component could pick the best scheduler on the fly when the workload changes. Instead of replacing the classical algorithms, I built a simple selector that looks at each batch of pending requests and decides which algorithm would perform best for that specific window. This matches the spirit of the assignment: add a small adaptive layer on top of classic OS concepts without building a complex machine learning system.

2 Implementation Summary

I wrote the entire simulator in Python. The core includes clean implementations of FCFS, SCAN, C-SCAN, and SSTF. All four algorithms receive the same request window and start from the same initial disk head position. Performance is measured by total seek distance — basically adding up how far the disk head has to move for every request in the window.

For the learned component, I used a decision tree from scikit-learn. I chose it because it is fast, lightweight, and easy to interpret, which fits the assignment’s goal perfectly. The model looks at several features from each request window: initial head position, mean and standard deviation of cylinder numbers, minimum and maximum cylinder, request range, average distance between consecutive requests, locality fraction, and a few others.

To create training labels, I ran all four classical schedulers on each window and picked the one with the lowest seek distance as the “best” choice. This gave the decision tree a clear target to learn from.

3 Experimental Design

The simulated disk uses cylinders 0 to 199. Each request window contains 60 requests. I generated four different workload patterns: sequential (mostly ordered requests with strong

locality), random (requests scattered across the disk), bursty (clusters of requests in a few areas), and mixed.

The real test uses a workload that shifts over time. The experiment is divided into three phases: the first phase is mostly sequential, the second switches to random, and the third becomes bursty. This setup is important because a scheduler that works well in one phase might struggle once the pattern changes. All algorithms and the learned selector were tested on exactly the same request windows, so the comparison stays fair.

4 Results

The decision tree achieved a held-out accuracy of 0.729. Its confidence was noticeably higher when it was right (average 0.849) than when it was wrong (average 0.652). This suggests the confidence score gives a useful but imperfect signal.

The table below shows total seek distance across the entire shifted workload (lower is better):

Method	Total Seek Distance
FCFS	43,594
SCAN	4,974
C-SCAN	6,383
SSTF	4,096
Learned Selector	3,862
Oracle Best	3,856

Table 1: Total seek distance comparison (lower is better).

The learned selector achieved the best result among all practical methods with a total seek distance of 3,862. It came extremely close to the theoretical oracle (3,856), which always picks the actual best algorithm for each window. That small gap shows the decision tree successfully learned useful patterns from the request window features.

5 Analysis and Discussion

FCFS performed terribly, with a total seek distance of 43,594. This wasn’t surprising. Because it strictly follows arrival order, it ends up making huge jumps across the disk once the workload turns random. It completely ignores spatial locality.

SSTF did very well overall (4,096), especially in bursty phases where requests are clustered together. The greedy approach works nicely when the head can serve many nearby requests before traveling far. However, it can still suffer when the current head position is unlucky or requests are spread out.

SCAN and C-SCAN fell in the middle. SCAN achieved 4,974 while C-SCAN got 6,383. Their directional sweeping reduces random back-and-forth movement, but they sometimes waste time traveling all the way to the disk edge even when there are no pending requests there.

The learned selector stood out because it didn’t stick to one algorithm for the whole experiment. It adapted its choice based on the current request window. This feels like the real advantage of learning-augmented heuristics: you keep the simple, understandable classical algorithms but add a lightweight layer that reacts to workload changes.

The confidence results were also interesting. The model was more confident when it made correct predictions, which is encouraging. Still, the calibration wasn’t perfect. In a real operating system, you would probably want fallback rules or periodic retraining before trusting such a component in production.

6 Conclusion

This project showed that even a simple learning-augmented approach can improve disk scheduling. I implemented FCFS, SCAN, C-SCAN, and SSTF, then added a decision tree that selects the best scheduler for each 60-request window using basic statistical features.

When tested under a workload that shifts from sequential to random to bursty behavior, the learned selector outperformed every fixed classical algorithm and came very close to the ideal oracle. The results support the idea that adaptive techniques can be valuable in operating systems, especially when workloads aren't stable.

At the same time, the experiment highlighted that these learned components should stay interpretable, be evaluated under changing conditions, and probably need safety mechanisms before being used in critical environments.

Overall, the project was a good balance between classical OS concepts and a small adaptive layer — exactly what the assignment asked for.

References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2014.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.